

Course Code	: BCS-131
Course Title	: Programming in C++
Assignment Number	: BCA_NEW(III)-131/Assignment/2025-26
Maximum Marks	: 100
Weightage	: 30%
Last Date for Submission	: 31th October 2025 (For July 2025 Session) 30th April, 2026 (For January 2026 Session)

This assignment has three questions carrying a total of 80 marks. Answer all the questions. Rest 20 marks are for viva-voce. You may use illustrations and diagrams to enhance explanations. Please go through the guidelines regarding assignments given in the Programme Guide for the format of presentation. Wherever required, you may write C++ program and take its output as part of solution.

Question 1. What are Virtual Constructor and Virtual Destructor? What are the differences between a Virtual Constructor and a Constructor? What are the differences between a Virtual Destructor and a Destructor? Explain with examples. (30 Marks)

Solution: Virtual Constructor and Virtual Destructor in C++

1. Virtual Constructor

Concept: In C++, a true "virtual constructor" does not exist. Constructors cannot be declared as virtual because when the constructor runs, the object does not exist, and thus no virtual table (vtable) or virtual pointer has been set up yet. The type of object to be created must be known at compile-time, which is against the nature of virtual functions that require run-time polymorphism.

Virtual Constructor Idiom (Workaround): To achieve similar polymorphic object creation, the *virtual constructor* design pattern is often used. This involves providing a virtual create() or clone() method in the base class. These methods return new objects of the correct derived type, chosen at run-time.

```
class Shape {
public:
    virtual ~Shape() { }
    virtual Shape* create() const = 0; //idiomatic "virtual constructor"
    virtual Shape* clone() const = 0; //idiomatic "virtual copy constructor"
};
class Circle : public Shape {
public:
    virtual Shape* create() const override { return new Circle(); }
    virtual Shape* clone() const override { return new Circle(*this); }
};
```

- **Advantages:**

- Enables polymorphic object creation at run-time.
- Supports extensibility and reduces coupling by letting client code work with base pointers.

2. Virtual Destructor

- **Concept:** A virtual destructor is declared in the base class with the virtual keyword. This ensures that when an object created through a base pointer is deleted, the destructor of the derived class is called first, followed by the base class destructor.

```
class Base {
public:
    virtual ~Base() {
        cout << "Base destructor called" << endl;
    }
};
class Derived : public Base {
public:
    ~Derived() {
        cout << "Derived destructor called" << endl;
    }
};
int main() {
    Base *b = new Derived();
    delete b; //invokes Derived destructor, then Base destructor
}
```

- **Why Use Virtual Destructor?**

- Prevents resource leaks and undefined behavior in polymorphic inheritance scenarios.
- Ensures proper cleanup of derived class resources when deleting objects through base pointers.

3. Differences Between Virtual Constructor and Constructor

Aspect	Constructor	Virtual Constructor (Idiom)
Declaration	Cannot be virtual	Achieved via virtual create() method
Purpose	Instantiate object of known class at compile-time	Instantiate derived objects at run-time

Polymorphism Support	No (compile-time type required)	Yes (enables run-time choice)
Actual Existence	Built-in language feature	Design pattern, not a language feature
When Called	At object creation	At run-time, via base pointer

4. Differences Between Virtual Destructor and Destructor

Aspect	Destructor	Virtual Destructor
Declaration	~ClassName()	virtual ~ClassName()
Behavior	Non-polymorphic	Polymorphic, supports inheritance
Use Case	Called for objects with static type	Required when deleting via base pointer
Cleanup	Only base class destructor called	Derived destructor called, then base class
Resource Management	May cause memory/resource leak	Prevents memory/resource leaks

5. Examples

- Virtual Constructor Idiom Example:

```
class Base {
public:
    virtual Base* create() { return new Base(); }
    virtual void display() { cout << "Base" << endl; }
};
class Derived : public Base {
public:
    virtual Base* create() override { return new Derived(); }
    virtual void display() override { cout << "Derived" << endl; }
};
Base* obj = new Derived();
Base* newObj = obj->create();
newObj->display(); // Output: Derived[7]
```

- Virtual Destructor Example:

```
cpp
class Base {
public:
    virtual ~Base() { cout << "Base destructor called" << endl; }
};
class Derived : public Base {
public:
```

```
~Derived() { cout << "Derived destructor called" << endl; }  
};  
Base* ptr = new Derived();  
delete ptr; // Output: Derived destructor called in Base destructor  
called[6][11][13]
```

6. Summary

- **Virtual Constructor:** Not allowed by language, used as design pattern for polymorphic object creation using virtual methods like create() or clone() in base class. Actual constructors can't be virtual
- **Virtual Destructor:** Declared using virtual keyword in base class, ensures correct destruction order in inheritance, prevents memory leaks and undefined behavior.
- **Difference:** Constructors initialize known types at compile-time, cannot be virtual due to language constraints. Virtual constructor idiom enables dynamic object creation. Destructors can (and should) be virtual in base classes for inheritance hierarchies; virtual destructors guarantee all destructors are called, while non-virtual destructors can lead to resource leaks.

Question 2. What do you mean by Exception Handling? How is it implemented? What are its advantages and disadvantages? (30 Marks)

Solution: Exception Handling in C++

1. Meaning of Exception Handling

Exception handling is a **mechanism in C++ to detect and manage runtime errors** in a structured way. Instead of causing the program to terminate unexpectedly, it allows you to handle errors gracefully, separating error-handling logic from regular code flow. Typical runtime errors, called exceptions, include division by zero, accessing invalid memory, and file I/O failures.

“Exception handling is a mechanism that separates code that detects and handles exceptional circumstances from the rest of your program.”

2. How Exception Handling is Implemented

Exception handling in C++ uses three main keywords: **try**, **catch**, and **throw**. Here's how it works:

- **try block:** Contains code that might cause an exception.
- **throw statement:** When an error occurs, the program uses throw to signal an exception.
- **catch block:** Contains code to handle the exception thrown by the try block. The program searches for a matching catch block.

Syntax

```
try {  
    // Code that may throw an exception  
    if (denominator == 0)  
        throw 0; // throwing an integer exception  
    divide = numerator / denominator;  
}  
catch (int num_exception) {  
    cout << "Error: Cannot divide by " << num_exception << endl;  
}
```

The flow: when an exception is thrown in the try block, the rest of the try block is skipped, and control moves to the catch block.

Example: Multiple Exceptions

```
try {  
    if (index >= 4)  
        throw "Error: Array out of bounds!"; //string literal  
    if (denominator == 0)  
        throw 0; //integer  
    arr[index] = numerator / denominator;  
}  
catch (const char* msg) {  
    cout << msg << endl;  
}  
catch (int num) {  
    cout << "Error: Cannot divide by " << num << endl;  
}  
catch (...) {  
    cout << "Unexpected exception!" << endl;  
}
```

This demonstrates handling different exceptions using multiple catch blocks.

3. Advantages of Exception Handling

- **Separation of Error Handling from Normal Code**
Try-catch blocks keep error-handling logic away from regular code, making the program easier to read and maintain.
- **Improved Program Structure**
Error-handling is centralized instead of scattered throughout the program, leading to cleaner and better-organized code.
- **Reduced Complexity & Flexibility**
Exception handling provides a unified mechanism to handle different errors, and developers can define custom exception types as required.

- **Better Debugging**
It makes locating and fixing runtime issues easier, saving development time.
- **Allows Functions to Handle Only Relevant Exceptions**
Unhandled exceptions are passed up the call stack, letting higher-level code handle unexpected errors.

4. Disadvantages of Exception Handling

- **Complexity**
Exception handling introduces more advanced concepts (throw, try, catch, exception classes, etc.) and can be harder for beginners compared to simple error codes.
- **Performance Costs**
Exception handling can slow down a program and increase its size, especially when an exception is actually thrown. The compiler adds extra code for stack unwinding, which can impact performance.
- **Invisible Control Flow**
Exceptions add multiple hidden exit points in functions, making code harder to trace, review, or debug. This can impact code maintainability for complex systems.
- **Unpredictability in Real-time Systems**
Exception handling isn't suitable for real-time systems where predictable code execution is required.

5. Summary Table

Aspect	Advantages	Disadvantages
Code Readability	Separation, clarity, maintainability	Complex control flow, harder to trace
Program Structure	Centralized error handling	Hidden exit points
Flexibility	Custom exception types, reusability	Complexity for beginners
Performance	Efficient in normal flow	Costly if exception thrown
Reliability	Safer, avoids abrupt termination	Not ideal for real-time systems

6. Conclusion

Exception handling in C++ is a powerful mechanism for managing runtime errors, implemented via **try, catch, and throw**. Its main advantages are clarity, maintainability, flexibility, and robust error management. However, it also brings complexity, possible performance overhead, and issues for real-time, highly constrained systems. Proper understanding and structured use are essential for leveraging its benefits in software development.

Question 3. What are the C++ header files that support the programs which draw figures? Make assumptions, if necessary. (20 Marks)

Solution: C++ Header Files That Support Drawing Figures

Several header files and libraries can be used in C++ programs for graphics and drawing figures. The most commonly used, especially in educational settings like BCA (IGNOU), is **graphics.h**. However, modern C++ programming also supports libraries like **winbgim.h**, **GL/glut.h**, and other third-party libraries such as SFML, SDL, and OpenGL.

1. graphics.h

- **Purpose:** Supports basic graphics functions such as drawing lines, circles, rectangles, polygons, and setting colors.
- **Commonly Used In:** Turbo C++, CodeBlocks (with winbgim port), Dev-C++.
- **Example Usage:**

```
#include <graphics.h>
int main()
{
    int gd = DETECT, gm;
    initgraph(&gd, &gm, "");
    circle(320, 240, 50); // Draws a circle
    rectangle(100, 100, 200, 200); // Draws a rectangle
    closegraph();
    return 0;
}
```

- **Functions Available:** initgraph(), closegraph(), circle(), rectangle(), line(), ellipse(), bar(), etc.
- **Note:** graphics.h is mostly supported on older compilers like Turbo C++; for modern setups, you may need the winbgim.h header and associated libraries.

2. winbgim.h

- **Purpose:** A Windows-compatible version of BGI graphics library, often used with CodeBlocks and Dev-C++.
- **Works With:** Modern IDEs instead of DOS-based Turbo C++.

3. Other Headers and Libraries

- **GL/glut.h** (OpenGL Utility Toolkit): Used in C++ for more advanced graphics and 3D visualization.
- **SFML/Graphics.hpp**: Modern cross-platform library for graphics, window handling, and multimedia.

- **SDL2/SDL.h:** Lightweight, fast library for graphics and game development.

4. Assumptions

Since IGNOU BCA often uses Turbo C++ or similar setups for teaching basic graphics, it is safe to assume that the graphics programming tasks are based on **graphics.h**. For modern environments, winbgim.h or other libraries may be required.

5. Summary Table

Header File	Purpose	Common Functions	Platform/Compiler
graphics.h	Basic 2D graphics operations (draw figures)	circle, line, rectangle	Turbo C++, CodeBlocks
winbgim.h	Windows port of graphics.h	Same as graphics.h	CodeBlocks, Dev-C++ (Windows)
GL/glut.h	Advanced 2D/3D graphics (OpenGL)	glutInit, glutDisplay	All platforms (OpenGL)
SFML/Graphics.hpp	Modern multimedia/graphics	draw, render	Modern C++ compilers
SDL2/SDL.h	Multimedia, graphics, input handling	SDL_RenderDrawLine	Cross-platform

6. Important Points

- **graphics.h** is most frequently used for academic purposes to draw figures.
- For newer compilers/environments, **winbgim.h** or third-party libraries are preferable.
- Always ensure the required header files and libraries are installed and linked in your development environment.